

Greedy Algorithms: Part 2

Dr. Bugra Caskurlu¹

¹Computer Engineering Department
TOBB ETU University

February, 2015

Greedy Algorithms

Definition and Proof Techniques

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.

Greedy Algorithms

Definition and Proof Techniques

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.

Greedy Algorithms

Definition and Proof Techniques

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- **It is easy to invent greedy algorithms for almost any problem, however; finding cases in which they work well, and proving that they work well, is the interesting challenge.**

Greedy Algorithms

Definition and Proof Techniques

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- **It is easy to invent greedy algorithms for almost any problem, however; finding cases in which they work well, and proving that they work well, is the interesting challenge.**
- There are two basic methods for proving that a greedy algorithm produces an optimal solution to a problem.

Greedy Algorithms

Definition and Proof Techniques

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- **It is easy to invent greedy algorithms for almost any problem, however; finding cases in which they work well, and proving that they work well, is the interesting challenge.**
- There are two basic methods for proving that a greedy algorithm produces an optimal solution to a problem.
- **Greedy Algorithm Stays Ahead:** Measures the greedy algorithm's progress in a step-by-step fashion, and prove that *it does better than any algorithm at each step*.

Greedy Algorithms

Definition and Proof Techniques

- An algorithm is **greedy** if it builds up a solution in small steps, choosing a decision at each step myopically to optimize some underlying criterion.
- One can often design many different greedy algorithms for the same problem, each one locally, incrementally optimizing some different measure on its way to a solution.
- **It is easy to invent greedy algorithms for almost any problem, however; finding cases in which they work well, and proving that they work well, is the interesting challenge.**
- There are two basic methods for proving that a greedy algorithm produces an optimal solution to a problem.
- **Greedy Algorithm Stays Ahead:** Measures the greedy algorithm's progress in a step-by-step fashion, and prove that *it does better than any algorithm at each step*.
- **Exchange argument:** Consider any possible solution to the problem and *gradually transforms it into the solution found by the greedy algorithm without hurting its quality*.

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$.

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$. A subset A of requests are compatible if all pairs of requests $i, j \in A$, $i \neq j$ are compatible.

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$. A subset A of requests are compatible if all pairs of requests $i, j \in A, i \neq j$ are compatible.
- The goal is to select a compatible subset of requests of maximum possible size.

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$. A subset A of requests are compatible if all pairs of requests $i, j \in A, i \neq j$ are compatible.
- The goal is to select a compatible subset of requests of maximum possible size. Compatible sets of maximum size will be called *optimum*.

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$. A subset A of requests are compatible if all pairs of requests $i, j \in A, i \neq j$ are compatible.
- The goal is to select a compatible subset of requests of maximum possible size. Compatible sets of maximum size will be called *optimum*.
- **Greedy Algorithm:** *Accept the request that finishes first among all requests that are compatible with previously accepted requests.*

Interval Scheduling: The Greedy Algorithm Stays Ahead

Formal Definition

- There will be n requests labeled $1, 2, \dots, n$, and each request i is specifying a start time $s(i)$ and a finish time $f(i)$. We have $s(i) < f(i)$ for all i .
- Two requests i and j are *compatible* if the requested intervals do not overlap: that is, either $f(i) \leq s(j)$, or $f(j) \leq s(i)$. A subset A of requests are compatible if all pairs of requests $i, j \in A, i \neq j$ are compatible.
- The goal is to select a compatible subset of requests of maximum possible size. Compatible sets of maximum size will be called *optimum*.
- **Greedy Algorithm:** *Accept the request that finishes first among all requests that are compatible with previously accepted requests.*
- Our proof used the *greedy algorithm stays ahead* property. Specifically, we proved that, **For all indices $r \leq k$, we have $f(i_r) \leq f(j_r)$** , where i_r is the r^{th} earliest finishing job accepted by our algorithm, and j_r is that of an optimal solution.

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time** $O(n \cdot \lg n)$.

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time** $O(n \cdot \lg n)$.
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$.

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time $O(n \cdot \lg n)$.**
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$. Thus, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. **This part takes $O(n)$ time.**

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time $O(n \cdot \lg n)$.**
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$. Thus, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. **This part takes $O(n)$ time.**
- Hence, **our algorithm runs in $O(n \cdot \lg n) + O(n) = O(n \cdot \lg n)$ time.**

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time $O(n \cdot \lg n)$.**
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$. Thus, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. **This part takes $O(n)$ time.**
- Hence, **our algorithm runs in $O(n \cdot \lg n) + O(n) = O(n \cdot \lg n)$ time.**

Variant: Scheduling All Intervals

- **Interval Partitioning (Coloring) Problem:** We have many identical resources and we wish to schedule *all* the requests using as few resources as possible.

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time $O(n \cdot \lg n)$.**
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$. Thus, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. **This part takes $O(n)$ time.**
- Hence, **our algorithm runs in $O(n \cdot \lg n) + O(n) = O(n \cdot \lg n)$ time.**

Variant: Scheduling All Intervals

- **Interval Partitioning (Coloring) Problem:** We have many identical resources and we wish to schedule *all* the requests using as few resources as possible.
- **Lower Bound:** *In any instance of the Interval Partitioning Problem, the number of resources needed is at least the depth (the maximum number of requests at any single point in the time-line) of the set of intervals.*

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time $O(n \cdot \lg n)$.**
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$. Thus, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. **This part takes $O(n)$ time.**
- Hence, **our algorithm runs in $O(n \cdot \lg n) + O(n) = O(n \cdot \lg n)$ time.**

Variant: Scheduling All Intervals

- **Interval Partitioning (Coloring) Problem:** We have many identical resources and we wish to schedule *all* the requests using as few resources as possible.
- **Lower Bound:** *In any instance of the Interval Partitioning Problem, the number of resources needed is at least the depth (the maximum number of requests at any single point in the time-line) of the set of intervals.*
- **Greedy Algorithm Matching the Lower Bound:** Sort the intervals in their start times. Assign each interval to an available resource.

Implementation, Running Time and Variants

Implementation and Running Time

- We begin by sorting the n requests in order of finishing time and labeling them in this order. **This takes time $O(n \cdot \lg n)$.**
- If the most recent interval we've selected ends at time f , we continue iterating through subsequent intervals until we reach the first j for which $s(j) \geq f$. Thus, we implement the greedy algorithm in one pass through the intervals, spending constant time per interval. **This part takes $O(n)$ time.**
- Hence, **our algorithm runs in $O(n \cdot \lg n) + O(n) = O(n \cdot \lg n)$ time.**

Variant: Scheduling All Intervals

- **Interval Partitioning (Coloring) Problem:** We have many identical resources and we wish to schedule *all* the requests using as few resources as possible.
- **Lower Bound:** *In any instance of the Interval Partitioning Problem, the number of resources needed is at least the depth (the maximum number of requests at any single point in the time-line) of the set of intervals.*
- **Greedy Algorithm Matching the Lower Bound:** Sort the intervals in their start times. Assign each interval to an available resource. This algorithm never uses more than d (depth of the set of intervals) resources, and thus optimal.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.
- More precisely, the cost of licence j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if licence j is purchased t months from now, it will cost $100 \cdot r_j^t$.

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.
- More precisely, the cost of licence j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if licence j is purchased t months from now, it will cost $100 \cdot r_j^t$.
- We will assume that the price growth rates are distinct, i.e., $r_i \neq r_j$ for licences $i \neq j$ (even though they start at the same price of 100 TL).

A Simple Example

Problem Definition

- Your friends are starting a security company that needs to obtain licences for n different pieces of cryptographic software. Due to regulations, they can only obtain these licences at the rate of at most one per month.
- Each licence is currently selling for a price of 100 TL. However, they are all becoming more expensive according to exponential growth curves.
- More precisely, the cost of licence j increases by a factor of $r_j > 1$ each month, where r_j is a given parameter. This means that if licence j is purchased t months from now, it will cost $100 \cdot r_j^t$.
- We will assume that the price growth rates are distinct, i.e., $r_i \neq r_j$ for licences $i \neq j$ (even though they start at the same price of 100 TL).
- **Question:** *In which order should the company buy the licences so that the total amount of money it spends is as small as possible?*

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time.

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s .

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s .
- Each request is now more flexible. Instead of a start time and finish time, the request i has a deadline d_i , and it requires a contiguous time interval of length t_i .

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s .
- Each request is now more flexible. Instead of a start time and finish time, the request i has a deadline d_i , and it requires a contiguous time interval of length t_i .
- Each request i must be assigned an interval of time of length t_i , and different requests must be assigned nonoverlapping intervals.

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s .
- Each request is now more flexible. Instead of a start time and finish time, the request i has a deadline d_i , and it requires a contiguous time interval of length t_i .
- Each request i must be assigned an interval of time of length t_i , and different requests must be assigned nonoverlapping intervals.
- Ideally, we would like to assign interval i a time slot before d_i , however, such an assignment may not always be possible. So, we are allowed to miss the deadline of some of the requests.

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s .
- Each request is now more flexible. Instead of a start time and finish time, the request i has a deadline d_i , and it requires a contiguous time interval of length t_i .
- Each request i must be assigned an interval of time of length t_i , and different requests must be assigned nonoverlapping intervals.
- Ideally, we would like to assign interval i a time slot before d_i , however, such an assignment may not always be possible. So, we are allowed to miss the deadline of some of the requests.
- We say that a request i is **late** if it misses the deadline (i.e., $f(i) > d_i$). The lateness of a request is defined to be $l_i = f(i) - d_i$, if the request is late. Otherwise (if, $f(i) \leq d_i$), $l_i = 0$.

Scheduling to Minimize Lateness

Problem Definition

- We have a single resource, and a set of n requests to use the resource for an interval of time. Assume that the resource is available starting at time s .
- Each request is now more flexible. Instead of a start time and finish time, the request i has a deadline d_i , and it requires a contiguous time interval of length t_i .
- Each request i must be assigned an interval of time of length t_i , and different requests must be assigned nonoverlapping intervals.
- Ideally, we would like to assign interval i a time slot before d_i , however, such an assignment may not always be possible. So, we are allowed to miss the deadline of some of the requests.
- We say that a request i is **late** if it misses the deadline (i.e., $f(i) > d_i$). The lateness of a request is defined to be $l_i = f(i) - d_i$, if the request is late. Otherwise (if, $f(i) \leq d_i$), $l_i = 0$.
- The goal in our new optimization problem is to minimize the maximum lateness, $L = \max_i l_i$.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows:
 $\{(1, 2), (2, 4), (3, 6)\}$.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1, 2), (2, 4), (3, 6)\}$. What is an optimal scheduling?

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1, 2), (2, 4), (3, 6)\}$. What is an optimal scheduling? Scheduling the jobs in the order 1, 2, 3 is the unique optimal scheduling, which incurs a maximum lateness of 0.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1, 2), (2, 4), (3, 6)\}$. What is an optimal scheduling? Scheduling the jobs in the order 1, 2, 3 is the unique optimal scheduling, which incurs a maximum lateness of 0.

Some Greedy Algorithms

- **Shortest Job First:** Schedule the jobs in order of increasing length t_i , so as to get the short jobs out of the way quickly.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1, 2), (2, 4), (3, 6)\}$. What is an optimal scheduling? Scheduling the jobs in the order 1, 2, 3 is the unique optimal scheduling, which incurs a maximum lateness of 0.

Some Greedy Algorithms

- **Shortest Job First:** Schedule the jobs in order of increasing length t_i , so as to get the short jobs out of the way quickly.
- It does not work for the set of requests $\{(1, 100), (10, 10)\}$.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1,2), (2,4), (3,6)\}$. What is an optimal scheduling? Scheduling the jobs in the order 1,2,3 is the unique optimal scheduling, which incurs a maximum lateness of 0.

Some Greedy Algorithms

- **Shortest Job First:** Schedule the jobs in order of increasing length t_i , so as to get the short jobs out of the way quickly.
- It does not work for the set of requests $\{(1,100), (10,10)\}$.
- **Minimum Slack Time First:** Let t be the current time. Schedule the job for which $(d_i - t) - t_i$ is minimized.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1,2), (2,4), (3,6)\}$. What is an optimal scheduling? Scheduling the jobs in the order 1,2,3 is the unique optimal scheduling, which incurs a maximum lateness of 0.

Some Greedy Algorithms

- **Shortest Job First:** Schedule the jobs in order of increasing length t_i , so as to get the short jobs out of the way quickly.
- It does not work for the set of requests $\{(1,100), (10,10)\}$.
- **Minimum Slack Time First:** Let t be the current time. Schedule the job for which $(d_i - t) - t_i$ is minimized.
- It does not work for the set of requests $\{(1,2), (10,10)\}$. This algorithm will select the jobs in the order 2,1, and the maximum lateness will be 9. However, if we selected the intervals in the order 1,2, the maximum lateness would be 1.

Example and a Greedy Algorithm Attempt

Example

Assume there are three requests, each of which is given as (t_i, d_i) pairs as follows: $\{(1,2), (2,4), (3,6)\}$. What is an optimal scheduling? Scheduling the jobs in the order 1,2,3 is the unique optimal scheduling, which incurs a maximum lateness of 0.

Some Greedy Algorithms

- **Shortest Job First:** Schedule the jobs in order of increasing length t_i , so as to get the short jobs out of the way quickly.
- It does not work for the set of requests $\{(1,100), (10,10)\}$.
- **Minimum Slack Time First:** Let t be the current time. Schedule the job for which $(d_i - t) - t_i$ is minimized.
- It does not work for the set of requests $\{(1,2), (10,10)\}$. This algorithm will select the jobs in the order 2, 1, and the maximum lateness will be 9. However, if we selected the intervals in the order 1, 2, the maximum lateness would be 1.
- **Earliest Deadline First:** At each step, select the job with the earliest deadline.

Pseudocode and Notation

Pseudocode

Order the jobs in order of their deadlines

Initially, $f = s$

Consider the jobs $i = 1, \dots, n$ in this order

Assign job i to interval from $s(i) = f$ to $f(i) = f + t_i$

Let $f = f + t_i$

Return the set of scheduled intervals $[s(i), f(i)] \forall i \in \{1, \dots, n\}$

Notation and Explanation

- By renaming the jobs if necessary, we assume that the jobs are labeled in the order of their deadlines, i.e., we have $d_1 \leq \dots \leq d_n$.
- We will schedule the jobs in this order. Job 1 will start at time $s = s(1)$, and end at time $f(1) = s(1) + t_1$; Job 2 will start at time $s(2) = f(1)$, and end time $f(2) = s(2) + t_2$, and so forth.
- We will use f to denote the finishing time of the last scheduled job (i.e., $f = f(n)$) when the algorithm terminates.

Proof Strategy

Strategy

- How can we prove that our schedule is optimal, i.e., its maximum lateness is as small as possible?

Proof Strategy

Strategy

- How can we prove that our schedule is optimal, i.e., its maximum lateness is as small as possible?
- **Exchange Argument:** We will start by an optimal schedule O . We will gradually modify O , preserving its optimality at each step, but eventually transforming it into A .

Proof Strategy

Strategy

- How can we prove that our schedule is optimal, i.e., its maximum lateness is as small as possible?
- **Exchange Argument:** We will start by an optimal schedule O . We will gradually modify O , preserving its optimality at each step, but eventually transforming it into A .

Lemma

A is a schedule with no idle time. Moreover, there is an optimal schedule with no idle time.

- └ Exchange Argument
- └ Scheduling to Minimize Lateness: An Exchange Argument

Definition and Lemma

Definition

A schedule A' has an ***inversion*** if a job with deadline d_i is scheduled before another job j with earlier deadline $d_j < d_i$.

Definition and Lemma

Definition

A schedule A' has an ***inversion*** if a job with deadline d_i is scheduled before another job j with earlier deadline $d_j < d_i$. Notice that the schedule A produced by our greedy algorithm has no inversions.

Definition and Lemma

Definition

A schedule A' has an ***inversion*** if a job with deadline d_i is scheduled before another job j with earlier deadline $d_j < d_i$. Notice that the schedule A produced by our greedy algorithm has no inversions.

Lemma

All schedules with no inversions and no idle time have the same maximum lateness.

Definition and Lemma

Definition

A schedule A' has an ***inversion*** if a job with deadline d_i is scheduled before another job j with earlier deadline $d_j < d_i$. Notice that the schedule A produced by our greedy algorithm has no inversions.

Lemma

All schedules with no inversions and no idle time have the same maximum lateness.

Proof

- If two different schedules have neither inversions nor idle time, they can only differ in the order in which jobs with identical deadlines are scheduled.

Definition and Lemma

Definition

A schedule A' has an ***inversion*** if a job with deadline d_i is scheduled before another job j with earlier deadline $d_j < d_i$. Notice that the schedule A produced by our greedy algorithm has no inversions.

Lemma

All schedules with no inversions and no idle time have the same maximum lateness.

Proof

- If two different schedules have neither inversions nor idle time, they can only differ in the order in which jobs with identical deadlines are scheduled.
- Consider such a deadline d . In both schedules, the jobs with deadline d are all scheduled consecutively. Among the jobs with deadline d , the last one has the maximum lateness, and this lateness does not depend on the order of the jobs. □

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.
- If we swap i and j , we get a schedule with one less inversion.

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.
- If we swap i and j , we get a schedule with one less inversion. All we need to show is that swapping a pair of consecutive jobs do not increase maximum lateness.

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.
- If we swap i and j , we get a schedule with one less inversion. All we need to show is that swapping a pair of consecutive jobs do not increase maximum lateness.
- In schedule O , each request is scheduled for the time interval $[s(r), f(r)]$, and has lateness l'_r . Let $L' = \max_r l'_r$.

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.
- If we swap i and j , we get a schedule with one less inversion. All we need to show is that swapping a pair of consecutive jobs do not increase maximum lateness.
- In schedule O , each request is scheduled for the time interval $[s(r), f(r)]$, and has lateness l'_r . Let $L' = \max_r l'_r$.
- Let $\bar{O}$ be the schedule, which is same as O , except that i and j are swapped. Define $\bar{s}(r), \bar{f}(r), \bar{l}_r$, and $\bar{L}$ to denote the corresponding quantities for the schedule $\bar{O}$.

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.
- If we swap i and j , we get a schedule with one less inversion. All we need to show is that swapping a pair of consecutive jobs do not increase maximum lateness.
- In schedule O , each request is scheduled for the time interval $[s(r), f(r)]$, and has lateness l'_r . Let $L' = \max_r l'_r$.
- Let $\overline{O}$ be the schedule, which is same as O , except that i and j are swapped. Define $\overline{s}(r), \overline{f}(r), \overline{l}_r$, and $\overline{L}$ to denote the corresponding quantities for the schedule $\overline{O}$.
- All jobs other than i and j finish at the same time in O and $\overline{O}$. Job j finishes even earlier in $\overline{O}$ than in O . Thus, we should only worry about job i .

Theorem and Proof

Theorem

There is an optimal schedule that has no inversions and no idle time.

Proof

- If O has an inversion, then there is a pair of jobs i and j such that j is scheduled immediately after i and has $d_j < d_i$.
- If we swap i and j , we get a schedule with one less inversion. All we need to show is that swapping a pair of consecutive jobs do not increase maximum lateness.
- In schedule O , each request is scheduled for the time interval $[s(r), f(r)]$, and has lateness l'_r . Let $L' = \max_r l'_r$.
- Let $\bar{O}$ be the schedule, which is same as O , except that i and j are swapped. Define $\bar{s}(r), \bar{f}(r), \bar{l}_r$, and $\bar{L}$ to denote the corresponding quantities for the schedule $\bar{O}$.
- All jobs other than i and j finish at the same time in O and $\bar{O}$. Job j finishes even earlier in $\bar{O}$ than in O . Thus, we should only worry about job i .
- **Crucial Point:** Job i cannot be more late in the schedule $\bar{O}$ than j was in the schedule O . Specifically, since $d_i > d_j$, we have $\bar{l}_i = \bar{f}(i) - d_i = f(j) - d_i < f(j) - d(j) = l'_j$. □

Optimal Caching

Problem Definition

- Consider a set U of n pieces of data stored in main memory.

Optimal Caching

Problem Definition

- Consider a set U of n pieces of data stored in main memory. We also have a faster memory, the *cache*, that can hold $k < n$ pieces of data at any one time.

Optimal Caching

Problem Definition

- Consider a set U of n pieces of data stored in main memory. We also have a faster memory, the *cache*, that can hold $k < n$ pieces of data at any one time.
- We will assume that the cache initially holds some set of k items. A sequence of data items $D = d_1, d_2, \dots, d_m$ drawn from U is presented to us. This is the sequence of memory references we must process, and in processing them we must decide at all times which k items to keep in the cache.

Optimal Caching

Problem Definition

- Consider a set U of n pieces of data stored in main memory. We also have a faster memory, the *cache*, that can hold $k < n$ pieces of data at any one time.
- We will assume that the cache initially holds some set of k items. A sequence of data items $D = d_1, d_2, \dots, d_m$ drawn from U is presented to us. This is the sequence of memory references we must process, and in processing them we must decide at all times which k items to keep in the cache.
- When item d_i is presented, we can access it very quickly if it is already in the cache; otherwise, we are required to bring it from main memory into the cache, and *evict* some other piece of data that is currently in the cache to make room for d_i . This is called a *cache miss*, and we want to have as few of them as possible.

Optimal Caching

Problem Definition

- Consider a set U of n pieces of data stored in main memory. We also have a faster memory, the *cache*, that can hold $k < n$ pieces of data at any one time.
- We will assume that the cache initially holds some set of k items. A sequence of data items $D = d_1, d_2, \dots, d_m$ drawn from U is presented to us. This is the sequence of memory references we must process, and in processing them we must decide at all times which k items to keep in the cache.
- When item d_i is presented, we can access it very quickly if it is already in the cache; otherwise, we are required to bring it from main memory into the cache, and *evict* some other piece of data that is currently in the cache to make room for d_i . This is called a *cache miss*, and we want to have as few of them as possible.

Example

- Suppose we have 3 items $\{a, b, c\}$, the cache size is $k = 2$, initial cache contents is $\{a, b\}$, and we are presented with the sequence a, b, c, b, c, a, b .

Optimal Caching

Problem Definition

- Consider a set U of n pieces of data stored in main memory. We also have a faster memory, the *cache*, that can hold $k < n$ pieces of data at any one time.
- We will assume that the cache initially holds some set of k items. A sequence of data items $D = d_1, d_2, \dots, d_m$ drawn from U is presented to us. This is the sequence of memory references we must process, and in processing them we must decide at all times which k items to keep in the cache.
- When item d_i is presented, we can access it very quickly if it is already in the cache; otherwise, we are required to bring it from main memory into the cache, and *evict* some other piece of data that is currently in the cache to make room for d_i . This is called a *cache miss*, and we want to have as few of them as possible.

Example

- Suppose we have 3 items $\{a, b, c\}$, the cache size is $k = 2$, initial cache contents is $\{a, b\}$, and we are presented with the sequence a, b, c, b, c, a, b .
- **Optimal Solution:** At step 3: bring c , evict a . At step 6: bring a , evict c .

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested.

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested. In general, one could imagine an algorithm that produces schedules that are not reduced, by bringing in items in steps they are not requested.

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested. In general, one could imagine an algorithm that produces schedules that are not reduced, by bringing in items in steps they are not requested.

Lemma

For every schedule S , there is a corresponding schedule $\bar{S}$ such that $\bar{S}$ is a reduced schedule, and $\bar{S}$ brings in at most as many items as the schedule S .

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested. In general, one could imagine an algorithm that produces schedules that are not reduced, by bringing in items in steps they are not requested.

Lemma

For every schedule S , there is a corresponding schedule $\bar{S}$ such that $\bar{S}$ is a reduced schedule, and $\bar{S}$ brings in at most as many items as the schedule S .

Proof

Let S be a schedule that may not be reduced.

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested. In general, one could imagine an algorithm that produces schedules that are not reduced, by bringing in items in steps they are not requested.

Lemma

For every schedule S , there is a corresponding schedule $\bar{S}$ such that $\bar{S}$ is a reduced schedule, and $\bar{S}$ brings in at most as many items as the schedule S .

Proof

Let S be a schedule that may not be reduced. In any step i where S brings in an item d that has not been requested, $\bar{S}$ leaves d in main memory.

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested. In general, one could imagine an algorithm that produces schedules that are not reduced, by bringing in items in steps they are not requested.

Lemma

For every schedule S , there is a corresponding schedule $\bar{S}$ such that $\bar{S}$ is a reduced schedule, and $\bar{S}$ brings in at most as many items as the schedule S .

Proof

Let S be a schedule that may not be reduced. In any step i where S brings in an item d that has not been requested, $\bar{S}$ leaves d in main memory. It only brings d into the cache in step $j > i$, in which d is requested.

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Greedy Algorithm and Auxiliary Lemmas

Farthest-in-Future Algorithm

When d_i needs to be brought in the cache, evict the item that is needed farthest into the future.

Definition: Reduced Schedule

A cache maintenance schedule is called a *reduced schedule* (that does the minimum amount of work necessary in a given step), if it only brings an item d into the cache at the step it is requested. In general, one could imagine an algorithm that produces schedules that are not reduced, by bringing in items in steps they are not requested.

Lemma

For every schedule S , there is a corresponding schedule $\bar{S}$ such that $\bar{S}$ is a reduced schedule, and $\bar{S}$ brings in at most as many items as the schedule S .

Proof

Let S be a schedule that may not be reduced. In any step i where S brings in an item d that has not been requested, $\bar{S}$ leaves d in main memory. It only brings d into the cache in step $j > i$, in which d is requested. In this way, the cache miss incurred by $\bar{S}$ in step j can be charged to the earlier cache operation performed by S in step i , when it brought in d . □

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm.

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm. Let S be a reduced *schedule that makes the same eviction decisions as S_{FF} through the first j items in the sequence for a number j . Then there is a reduced schedule S' that makes the same eviction decisions as S_{FF} through the first $(j + 1)$ items, and incurs no more misses than S does.*

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm. Let S be a reduced *schedule that makes the same eviction decisions as S_{FF} through the first j items in the sequence for a number j . Then there is a reduced schedule S' that makes the same eviction decisions as S_{FF} through the first $(j+1)$ items, and incurs no more misses than S does.*

Proof of Simple Cases

- Consider the $(j+1)^{th}$ request, to item $d = d_{j+1}$.

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm. Let S be a reduced *schedule that makes the same eviction decisions as S_{FF} through the first j items in the sequence for a number j . Then there is a reduced schedule S' that makes the same eviction decisions as S_{FF} through the first $(j+1)$ items, and incurs no more misses than S does.*

Proof of Simple Cases

- Consider the $(j+1)^{th}$ request, to item $d = d_{j+1}$. Since S and S_{FF} have agreed up to this point, they have the same cache contents.

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm. Let S be a reduced *schedule that makes the same eviction decisions as S_{FF} through the first j items in the sequence for a number j . Then there is a reduced schedule S' that makes the same eviction decisions as S_{FF} through the first $(j+1)$ items, and incurs no more misses than S does.*

Proof of Simple Cases

- Consider the $(j+1)^{th}$ request, to item $d = d_{j+1}$. Since S and S_{FF} have agreed up to this point, they have the same cache contents.
- **If d is in the cache for both:** No eviction decision is necessary and so S agrees with S_{FF} through step $(j+1)$. Thus, $S' = S$.

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm. Let S be a reduced *schedule that makes the same eviction decisions as S_{FF} through the first j items in the sequence for a number j . Then there is a reduced schedule S' that makes the same eviction decisions as S_{FF} through the first $(j+1)$ items, and incurs no more misses than S does.*

Proof of Simple Cases

- Consider the $(j+1)^{th}$ request, to item $d = d_{j+1}$. Since S and S_{FF} have agreed up to this point, they have the same cache contents.
- **If d is in the cache for both:** No eviction decision is necessary and so S agrees with S_{FF} through step $(j+1)$. Thus, $S' = S$.
- **If d is not in the cache, but both S and S_{FF} evict the same item to make room for d :** $S' = S$.

Proving the Optimality of Farthest-in-Future

Lemma

Consider an arbitrary sequence D of memory references, and let S_{FF} denote the schedule produced by Farthest-in-Future algorithm. Let S be a reduced *schedule that makes the same eviction decisions as S_{FF} through the first j items in the sequence for a number j . Then there is a reduced schedule S' that makes the same eviction decisions as S_{FF} through the first $(j+1)$ items, and incurs no more misses than S does.*

Proof of Simple Cases

- Consider the $(j+1)^{th}$ request, to item $d = d_{j+1}$. Since S and S_{FF} have agreed up to this point, they have the same cache contents.
- **If d is in the cache for both:** No eviction decision is necessary and so S agrees with S_{FF} through step $(j+1)$. Thus, $S' = S$.
- **If d is not in the cache, but both S and S_{FF} evict the same item to make room for d :** $S' = S$.
- **Interesting Case:** d is not in the cache, S evicts j for d , and S_{FF} evicts $e \neq f$ for d .

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step.

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S .

- └ Exchange Argument
- └ Optimal Caching: A More Complex Exchange Argument

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time.

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .**

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .** Since S , and S' only differ on e and f , g is not in the cache of S' . So, S' evicts f for g , and have the same cache contents as S .

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .** Since S , and S' only differ on e and f , g is not in the cache of S' . So, S' evicts f for g , and have the same cache contents as S .
- **There is a request to f , and S evicts e for f .** S' does not need to make any eviction. It has a cache hit, and the contents of the cache is now the same as that of S .

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .** Since S , and S' only differ on e and f , g is not in the cache of S' . So, S' evicts f for g , and have the same cache contents as S .
- **There is a request to f , and S evicts e for f .** S' does not need to make any eviction. It has a cache hit, and the contents of the cache is now the same as that of S .
- **There is a request to f , and S evicts $e \neq e'$ for f .** Then S' evicts e' , and brings e from main memory. Then the cache contents are the same for both S and S' .

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .** Since S , and S' only differ on e and f , g is not in the cache of S' . So, S' evicts f for g , and have the same cache contents as S .
- **There is a request to f , and S evicts e for f .** S' does not need to make any eviction. It has a cache hit, and the contents of the cache is now the same as that of S .
- **There is a request to f , and S evicts $e \neq e'$ for f .** Then S' evicts e' , and brings e from main memory. Then the cache contents are the same for both S and S' . *(Are we done for this case as well, think a bit on that!)*

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .** Since S , and S' only differ on e and f , g is not in the cache of S' . So, S' evicts f for g , and have the same cache contents as S .
- **There is a request to f , and S evicts e for f .** S' does not need to make any eviction. It has a cache hit, and the contents of the cache is now the same as that of S .
- **There is a request to f , and S evicts $e \neq e'$ for f .** Then S' evicts e' , and brings e from main memory. Then the cache contents are the same for both S and S' . *(Are we done for this case as well, think a bit on that!)*
- **There is a request to e , and S have a cache hit.**

Continuing the Proof

Proof of Interesting Case

At step $(j + 1)$, S' evicts e rather than f , so it agrees with S_{FF} in the first $(j + 1)$ step. We also need to ensure that S' incurs no more misses than S . **Our strategy** in constructing S' will be that S' will get its cache back to the same state as S a.s.a.p, while not incurring unnecessary misses in the mean time. Specifically, S' behaves exactly like S until one of the following happens for the first time:

- **There is a request to an item $g \neq e, f$ that is not in the cache of S , and S evicts e for g .** Since S , and S' only differ on e and f , g is not in the cache of S' . So, S' evicts f for g , and have the same cache contents as S .
- **There is a request to f , and S evicts e for f .** S' does not need to make any eviction. It has a cache hit, and the contents of the cache is now the same as that of S .
- **There is a request to f , and S evicts $e \neq e'$ for f .** Then S' evicts e' , and brings e from main memory. Then the cache contents are the same for both S and S' . (*Are we done for this case as well, think a bit on that!*)
- **There is a request to e , and S have a cache hit.** There cannot be a request to e , before there is a request to f . Since otherwise, S_{FF} would not evict e at step $(j + 1)$.

Completing the Proof

Avoiding Pitfalls

- In case 3 above, there was a request to f , and S' evicted e' to bring e from main memory!

Completing the Proof

Avoiding Pitfalls

- In case 3 above, there was a request to f , and S' evicted e' to bring e from main memory!
- If case 3 happens before cases 1 or 2, we will still have a schedule S' that agrees with S_{FF} in the first $(j + 1)$ steps, and have no more misses than S . However, S' is not a reduced schedule.

Completing the Proof

Avoiding Pitfalls

- In case 3 above, there was a request to f , and S' evicted e' to bring e from main memory!
- If case 3 happens before cases 1 or 2, we will still have a schedule S' that agrees with S_{FF} in the first $(j + 1)$ steps, and have no more misses than S . However, S' is not a reduced schedule.
- So in this case, we will not report S' , but the schedule $\overline{S'}$, which is the reduced form of S' . □

Completing the Proof

Avoiding Pitfalls

- In case 3 above, there was a request to f , and S' evicted e' to bring e from main memory!
- If case 3 happens before cases 1 or 2, we will still have a schedule S' that agrees with S_{FF} in the first $(j + 1)$ steps, and have no more misses than S . However, S' is not a reduced schedule.
- So in this case, we will not report S' , but the schedule $\overline{S'}$, which is the reduced form of S' . □

Theorem

S_{FF} incurs no more misses than any other schedule S^* and hence it is optimal.

Completing the Proof

Avoiding Pitfalls

- In case 3 above, there was a request to f , and S' evicted e' to bring e from main memory!
- If case 3 happens before cases 1 or 2, we will still have a schedule S' that agrees with S_{FF} in the first $(j+1)$ steps, and have no more misses than S . However, S' is not a reduced schedule.
- So in this case, we will not report S' , but the schedule $\overline{S'}$, which is the reduced form of S' . □

Theorem

S_{FF} incurs no more misses than any other schedule S^* and hence it is optimal.

Proof

We begin with an optimal (reduced) schedule S^* , and use the above algorithm to construct a schedule S_1 that agrees with S_{FF} through the first step. We continue applying the same algorithm inductively for $j = 1, 2, \dots, m$, producing schedules S_j that agree with S_{FF} through the first j steps. Each schedule incurs no more misses than the previous one, and by definition $S_m = S_{FF}$ by definition, where $m = |D|$.

Next Lecture

Thursday

- Dijkstra's algorithm for finding shortest paths in directed graphs with nonnegative edge weights.

Next Lecture

Thursday

- Dijkstra's algorithm for finding shortest paths in directed graphs with nonnegative edge weights.
- Before that we will define graphs, see the representations of graphs.

Next Lecture

Thursday

- Dijkstra's algorithm for finding shortest paths in directed graphs with nonnegative edge weights.
- Before that we will define graphs, see the representations of graphs.
- And possibly (depending on the time) some elementary graph algorithms.